



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Análisis del modelo C4: Arquitectura de sistemas en 4 niveles

Constructos, notación, metamodelo y semántica de un lenguaje visual para describir arquitectura de software aplicado a un sistema ciberfísico.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Qué es el modelo C4?

Origen. Creado por Simon Brown (2006+) como respuesta a las limitaciones de UML para arquitectura de software. Hoy es estándar de facto en la industria.

Propósito. Describir la **arquitectura** de un sistema de software en **4 niveles de abstracción**, desde visión empresarial hasta código, facilitando comunicación entre stakeholders técnicos y no técnicos.

¿Por qué 4 niveles? Porque arquitectura no es una vista única. El contexto importa al CTO, los contenedores al DevOps, los componentes al arquitecto, el código al desarrollador.

LOS 4 NIVELES

Level 1: Context

Sistema + usuarios + sistemas externos

Level 2: Container

Aplicaciones, bases de datos, nodos físicos

Level 3: Component

Módulos funcionales dentro de un container

Level 4: Code

Clases, funciones, métodos

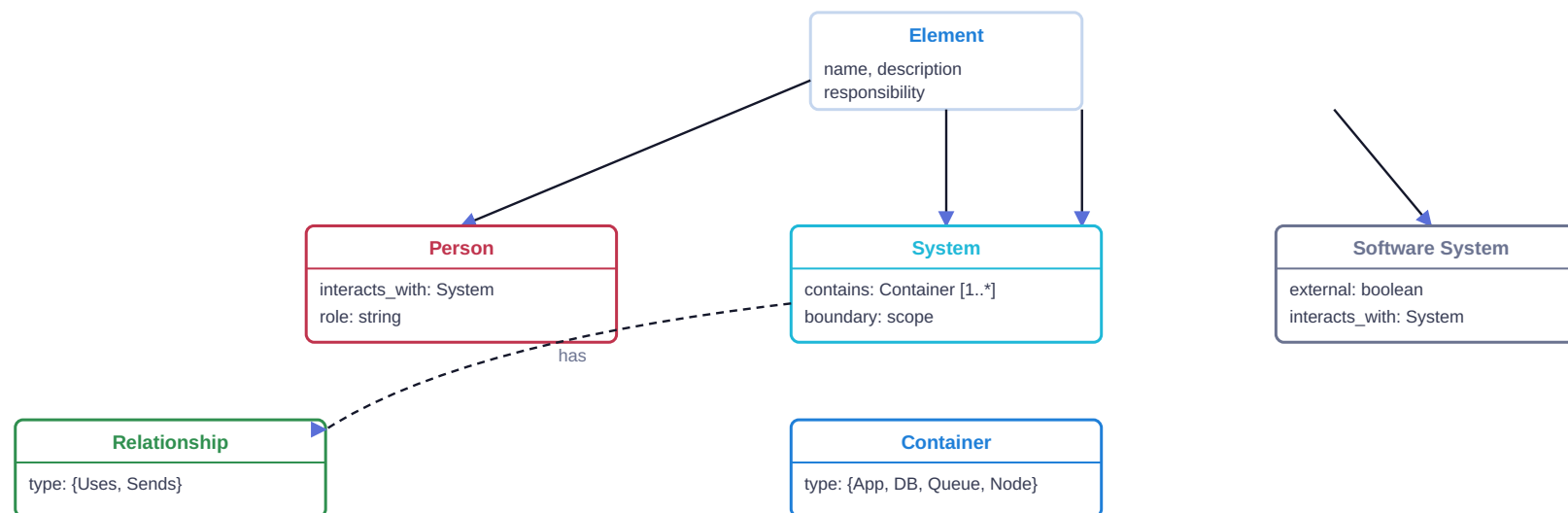
Constructos y conceptos del modelo C4

CONSTRUCTO	ROL
System	El software que se describe
Person	Usuario o actor externo
Software System	Sistema externo que interactúa
Container	Aplicación, BD, nodo, microservicio
Component	Módulo lógico dentro de un container
Code	Clases, métodos, funciones
Relationship	Interacción: Uses, Sends, Calls

RESTRICCIONES (BUENA FORMACIÓN)

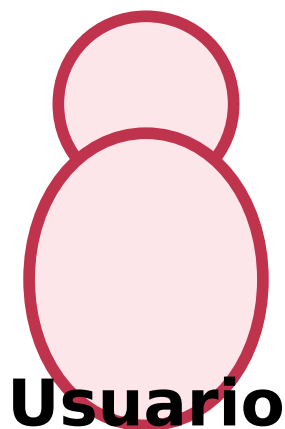
- Un **System** contiene ≥ 1 **Container**.
- Un **Container** contiene ≥ 1 **Component** (o es atómico).
- Un **Component** contiene ≥ 1 **Code unit**.
- Las relaciones **Uses** representan sincronía (HTTP, RPC).
- Las relaciones **Sends** representan asincronía (MQTT, EventBus).
- Solo se conectan elementos del **mismo nivel** (no Container → Code).

Relaciones entre constructos (diagrama de clases simplificado)



Notación visual estándar de C4

Person



System

Container

Face Monitor

[ESP32-CAM]

periódico 500ms

Relationship (MQTT)

¿Qué significa cada nivel? (Transformación de abstracción)

1 Context

¿Quiénes son los usuarios? ¿Qué otros sistemas existen? ¿Límites del sistema?

2 Container

¿Qué aplicaciones/nodos/DBs hay? ¿Cómo se despliegan? ¿Tecnología base?

3 Component

¿Qué módulos funcionales hay dentro? ¿Cómo se comunican? ¿Responsabilidades?

4 Code

¿Qué clases/métodos implementan? ¿Patrones de diseño? ¿Detalles técnicos?

EN SVIF:

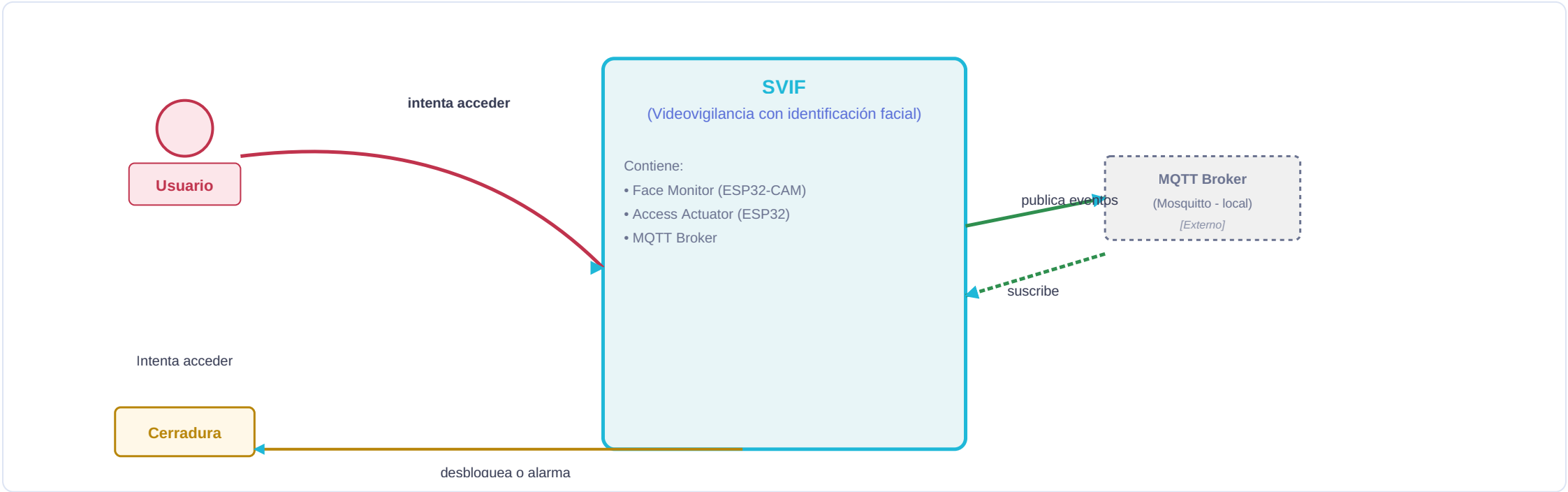
Context: Usuario quiere acceder, cámara detecta, cerradura se abre.

Container: Dos nodos ESP32 (Face Monitor + Access Actuator) comunicados por MQTT.

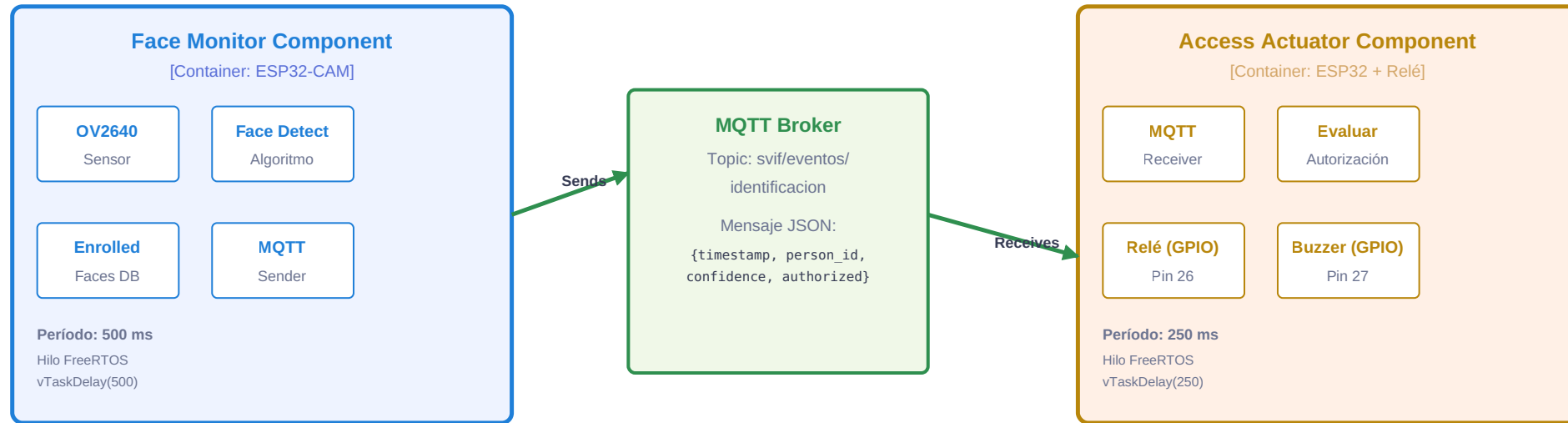
Component: Dentro de Face Monitor: captura, detección, identificación, MQTT sender.

Code: monitorearPresenciaTask(), capturarImagen(), identificarRostro(), publicIdentificationEvent().

LEVEL 1 Context: Sistema en contexto

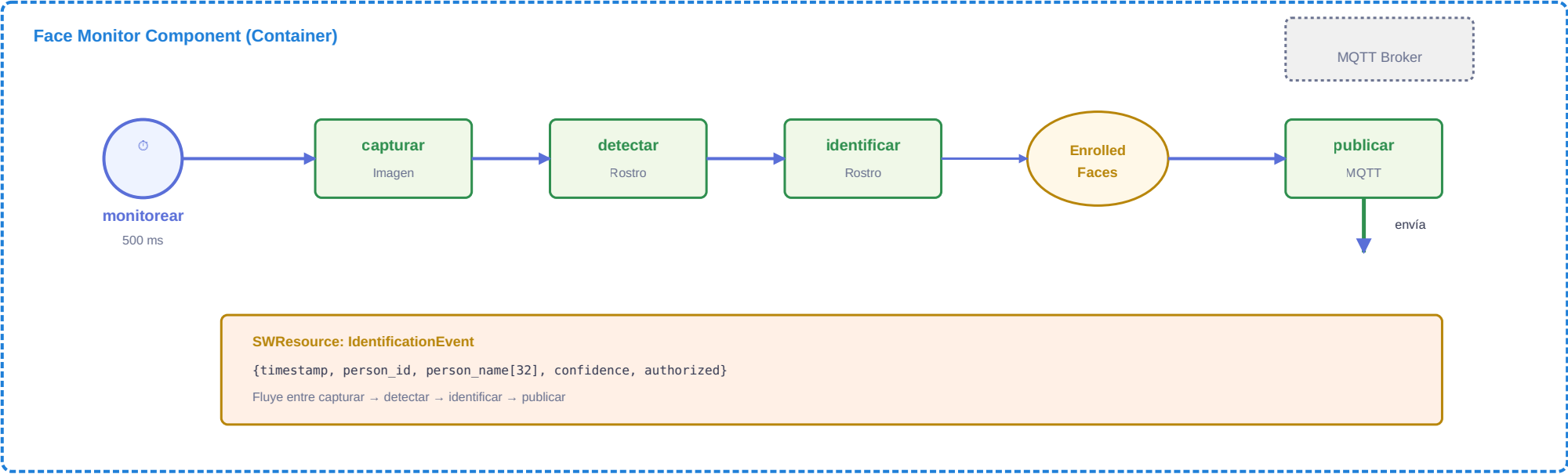


Container: Nodos y tecnología



LEVEL 3

Component: Dentro de Face Monitor



¿Por qué C4 es útil y cuáles son sus limitaciones para CPS?

FORTALEZAS DE C4

- **Escalable.** 4 niveles claros permiten comunicar desde CEO hasta desarrollador sin abrumar.
- **Agnóstico.** No asume tecnología; funciona igual para Java, Node, Go, IoT.
- **Composable.** Se puede expandir con diagramas adicionales (deployment, infraestructura).
- **Bidireccional.** El código puede generar C4 (inversión) o C4 guiar el código (forward-engineering).
- **Comunicación.** Diagramas visuales ayudan a stakeholders no técnicos a entender estructura.

LIMITACIONES PARA CPS

- **No muestra tiempo.** ¿Cada cuánto se ejecuta monitorear? (500 ms en SVIF) → necesita anotación.
- **No muestra transiciones.** ¿Qué ocurre si authorized=true vs false? → necesita lógica de control.
- **No muestra hardware.** ¿Qué pin? ¿Qué sensor? → apenas menciona HWResource.
- **No muestra restricciones temporales duras.** ¿Deadline? ¿Jitter tolerable? → requiere DSL más específico.
- **Abstracción variable.** Un "Container" puede ser app web o nodo ESP32 con 2KB RAM → demasiado genérico.

REFLEXIÓN: ¿C4 O DSL PIM?

C4 es excelente para arquitectura de software general. Comunica limpiamente "Face Monitor y Access Actuator son nodos separados, conectados por MQTT." Pero **CPS añade requisitos que C4 no contempla:** periodicidad, hardware específico, refinamientos de decisión (if authorized then unlock else alarm), trazabilidad a objetivos (iStar 2.0). Por eso en el proceso MDD4CPS, después de definir CIM (iStar) y Level 2 (C4), necesitamos un **DSL PIM especializado** que capture esos detalles antes de llegar a código.

Conclusión: C4 es el "contexto y contenedores"; DSL PIM es el "comportamiento y timing". Complementarios.

Fuentes consultadas

Brown, S. (2006–2024). *C4 Model*. <https://c4model.com>

Language and tool for visualizing software architecture at four levels of abstraction.

Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.

Comprehensive guide to designing and implementing DSLs; discusses C4 within broader modeling context.

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). An agent-oriented model-driven development process for cyber-physical systems. *CibSE 2025* (pp. 150–164). SBC.

Process combining iStar 2.0 (CIM), DSL PIM, and C++ PSM; includes C4 for Level 2 container view.

UML 2.5.1 Specification. (2017). Object Management Group (OMG). <https://www.omg.org/spec/UML/>
Standard for unified modeling language; C4 is positioned as simpler alternative for architecture views.